



Lab: Simulate GPU Provisioning and Monitoring with DCGM



Objective: Learn how to provision, monitor, and manage GPU resources using **NVIDIA Data Center GPU Manager (DCGM)** in a simulated or containerized environment. Ideal for learners without access to a full on-prem GPU cluster.



Lab Overview

You'll simulate GPU monitoring using:

- A local or Colab/Linux VM with **Docker + NVIDIA Container Toolkit**
- The **NVIDIA DCGM exporter container**
- Tools like `dogmi` CLI and Prometheus metrics viewer



What You Need

- A system with **NVIDIA GPU and drivers installed**, OR a **cloud VM** (e.g., AWS EC2 with GPU)
- **Docker** installed
- **NVIDIA Container Toolkit** (`nvidia-docker2`) installed
- Optional: **Prometheus/Grafana** or basic metrics viewer (`curl`, browser)



Step 1: Verify GPU Access

Make sure your system recognizes the GPU.

```
nvidia-smi
```



You should see your GPU model (e.g., A100, T4, etc.), utilization, and temperature.



Step 2: Install DCGM via Docker

Use the official **DCGM Docker container**, which contains:

- DCGM CLI (`dogmi`)
- DCGM Daemon (`dogmd`)
- DCGM Exporter for Prometheus

Run this in terminal:

```
docker run -d --rm \
  --gpus all \
  --name dcgm \
  --cap-add SYS_ADMIN \
  --volume /var/run/docker.sock:/var/run/docker.sock \
  nvcr.io/nvidia/dcgm:3.2.1
```

✅ This starts DCGM in detached mode (`-d`), accessing all GPUs.

Step 3: Connect to the Container

```
docker exec -it dcgm /bin/bash
```

Now you're inside the DCGM container where you can run GPU commands.

Step 4: Run Basic GPU Discovery

Inside the container, list all GPUs:

```
dcgmi discovery -l
```

✅ Output shows each GPU index, PCI bus, device name (e.g., GPU 0: NVIDIA A100).

Step 5: Check GPU Health

Run a basic health check:

```
dcgmi health -c
```

This command:

- Checks thermals
- ECC errors
- PCIe bandwidth
- Memory errors

✅ Output tells you PASS/FAIL status of each health check per GPU.

Step 6: Monitor Utilization & Memory

View live stats for a specific GPU (e.g., GPU 0):

```
dcgmi stats -i 0
```

You'll see:

- GPU utilization (%)
- Memory usage (MB)
- PCI throughput
- Power draw

✅ Try running this repeatedly to watch real-time changes.

Step 7: Simulate GPU Workload (Optional)

If you're on a GPU-capable system, open another terminal and run a basic workload like training a small model in PyTorch/TensorFlow or:

```
docker run --rm --gpus all nvidia/cuda:12.2.0-base nvidia-smi
```

Then re-run:

```
dcgmi stats -i 0
```

✅ You should see memory and SM occupancy increase under load.

Step 8: Enable GPU Policy Enforcement (Bonus)

Set a power cap policy (e.g., limit GPU to 150W):

```
dcgmi policy -i 0 --set -t 1 -d 150
```

This helps simulate **energy-aware scheduling** or **thermal protection**.

Reset it:

```
dcgmi policy -i 0 --clear
```

Step 9: View Metrics via Prometheus (Optional)

If you'd like to view metrics in a Prometheus-like format:

1. Expose port 9400 in the container:

```
docker run -d --rm \
  --gpus all \
```

```
-p 9400:9400 \  
--name dcgm \  
--cap-add SYS_ADMIN \  
--volume /var/run/docker.sock:/var/run/docker.sock \  
nvcr.io/nvidia/dcgm:3.2.1 dcgm-exporter
```

2. Visit: <http://localhost:9400/metrics> in a browser

✅ You'll see real-time GPU metrics in Prometheus format.

What You Learned

- How to install and run DCGM in a Docker container
 - How to check GPU health, utilization, and policy
 - How to simulate real GPU workloads
 - How DCGM integrates with monitoring and scheduling workflows
-